Worksheet 1 — Security Mindset & Threat Modeling (3 hrs)

> **Course:** Software Security (KOSEN69) · **Week 1 Aligned to:** OWASP 2025 A06 Insecure Design · CWE-501 (Trust Boundary Violation) **Signature game:** "Elevation of Privilege" (Microsoft STRIDE card deck)

> **Ethics note:** This week is *modeling only* — you analyze design, you do **not** attack the app. Run the sample app only on your own VM/localhost. Never apply these techniques to systems you do not own or lack written permission to test.

Part 1 — Student Information

Name	Student ID	Date	Group	AI Tool
Thura Aung	6631503094	15.8.2026	-	ChatGPT

Part 2 — Lecture Questions

Answer in your own words (2–4 sentences each).

1. Define the CIA triad and give one concrete failure example for each of the three properties.

- 1.CIA triad:The CIA triad stands for Confidentiality, Integrity, and Availability, the three core goals of information security. A confidentiality failure could be a hacker stealing private student records, an integrity failure could be an attacker changing exam grades, and an availability failure could be a website being taken offline by a DDoS attack.

2. What is a *trust boundary*, and why does data crossing one deserve extra scrutiny?

- 2.A trust boundary is a point where data or control moves between systems, components, or users with different levels of trust. Data crossing a trust boundary deserves extra scrutiny because it may be untrusted, manipulated, or malicious, so it should be validated and properly authorized before being accepted.

3. Explain "attack surface." Name two things that increase it in a web app.

- 3.An attack surface is the collection of all possible entry points that an attacker could use to compromise a system. In a web application, the attack surface can increase by adding more public APIs/endpoints or by using third-party libraries and services with potential vulnerabilities.

4. What does each STRIDE letter map to, and which security property does each threat violate?

- 4.STRIDE stands for Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. They correspond respectively to violations of authentication, integrity, non-repudiation/accountability, confidentiality, availability, and authorization.

5. What does "Secure by Design" (CISA) mean, and how does it differ from bolting security on after release?

- 5.Secure by Design, as promoted by CISA, means building security into a product from the beginning rather than treating it as an optional feature. Instead of releasing an insecure application and fixing vulnerabilities afterward, developers consider threats, secure defaults, and protections throughout design, development, testing, and maintenance.

Part 3 — Hands-on Lab (180 min)

Learning goals: build a data-flow diagram (DFD), apply STRIDE to a real Flask app, rank risks, and propose mitigations. **Prerequisites:** Docker + Docker Compose in your VM; a drawing tool (draw.io / paper + photo); the Elevation of Privilege deck (print or virtual) — free print-and-play PDF at github.com/adamshostack/eop.

Environment setup

```
cd labs/week01-threat-modeling
docker compose up --build           # starts sample-app on http://localhost:8080
curl -s -X POST localhost:8080/notes -H 'Content-Type: application/json' \
     -d '{"owner":"alice","body":"hello"}'   # observe behavior, do not attack
```

```
curl -s localhost:8080/notes

echo "demo file" > demo.txt
curl -s -X POST localhost:8080/upload -F "file=@demo.txt"   # observe behavior, do not attack
curl -s localhost:8080/files/demo.txt
```

Source to model lives in `sample-app/app.py`. Template to fill: `THREAT-MODEL-TEMPLATE.md` (copy it, do not edit the original).

What to submit per task: the threat/element identified + a screenshot (DFD, table, or running app) + a 2–3 sentence mitigation.

Task 0 — Onboarding (5 min) · *Goal:* prove the environment works. *Steps:* `docker compose up`, hit `/notes` and `/files/<name>`, read `sample-app/app.py`. *Deliverable:* screenshot of the running app + the JSON response.

(Task 0)

```
 ✔Image week01-threat-modeling-sample-app       Built                                                    16.7s
 ✔Network week01-threat-modeling_default        Created                                                   0.0s
 ✔Container week01-threat-modeling-sample-app-1 Created                                                   0.1s
Attaching to sample-app-1
sample-app-1  |  * Serving Flask app 'app'
sample-app-1  |  * Debug mode: off
sample-app-1  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
sample-app-1  |  * Running on all addresses (0.0.0.0)
sample-app-1  |  * Running on http://127.0.0.1:5000
sample-app-1  |  * Running on http://172.18.0.2:5000
sample-app-1  | Press CTRL+C to quit
sample-app-1  | 172.18.0.1 - - [15/Aug/2026 06:40:54] "POST /notes HTTP/1.1" 400 -
sample-app-1  | 172.18.0.1 - - [15/Aug/2026 06:41:50] "POST /notes HTTP/1.1" 200 -
sample-app-1  | 172.18.0.1 - - [15/Aug/2026 06:41:57] "GET /notes HTTP/1.1" 200 -
sample-app-1  | 172.18.0.1 - - [15/Aug/2026 06:43:02] "POST /upload HTTP/1.1" 200 -
sample-app-1  | 172.18.0.1 - - [15/Aug/2026 06:43:24] "GET /files/demo.txt HTTP/1.1" 200 -

v View in Docker Desktop   o View Config   w Enable Watch   d Detach
```

```
PS C:\Users\Acer Nitro5> Invoke-RestMethod `
>>     -Uri "http://localhost:8080/notes" `
>>     -Method POST `
>>     -ContentType "application/json" `
>>     -Body $body


Length         : 3
LongLength     : 3
Rank           : 1
SyncRoot       : {1, alice, hello}
IsReadOnly     : False
IsFixedSize    : True
IsSynchronized : False
Count          : 3



PS C:\Users\Acer Nitro5> Invoke-RestMethod -Uri "http://localhost:8080/notes" -Method GET


Length         : 3
LongLength     : 3
Rank           : 1
SyncRoot       : {1, alice, hello}
IsReadOnly     : False
IsFixedSize    : True
IsSynchronized : False
Count          : 3



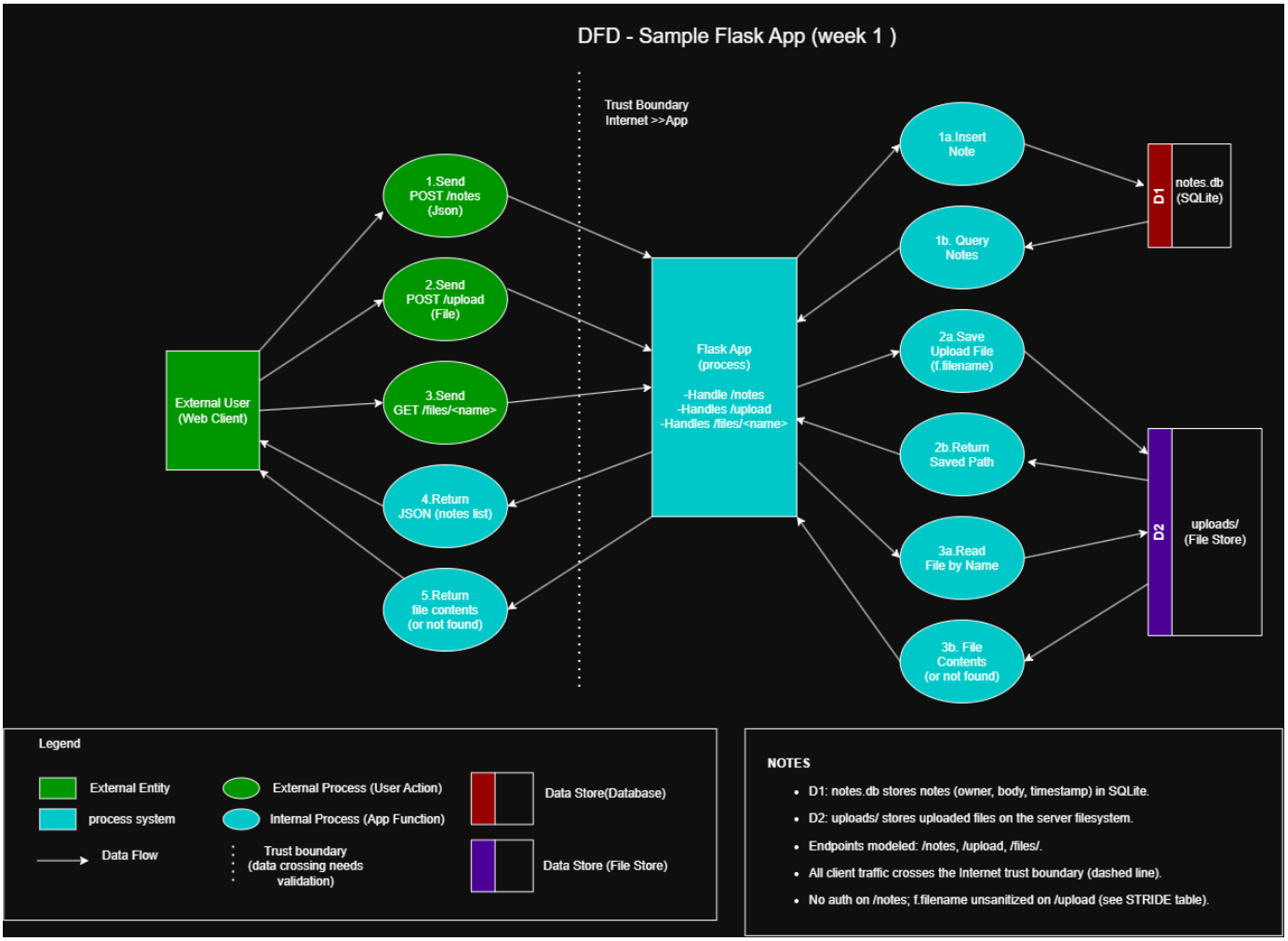
PS C:\Users\Acer Nitro5> echo "demo file" > demo.txt
PS C:\Users\Acer Nitro5> curl -s -X POST localhost:8080/upload -F "file=@demo.txt"
Invoke-WebRequest : A positional parameter cannot be found that accepts argument 'localhost:8080/upload'.
At line:1 char:1
+ curl -s -X POST localhost:8080/upload -F "file=@demo.txt"
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidArgument: (:) [Invoke-WebRequest], ParameterBindingException
    + FullyQualifiedErrorId : PositionalParameterNotFound,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

PS C:\Users\Acer Nitro5> curl.exe -s -X POST http://localhost:8080/upload `
>>   -F "file=@demo.txt"
{"saved":"demo.txt"}
PS C:\Users\Acer Nitro5> curl.exe -s http://localhost:8080/files/demo.txt
PS C:\Users\Acer Nitro5>
```

Task 1 — Draw the DFD (25 min) · *Goal:* map the system. *Steps:* identify the external entity (web client), the process (Flask app), the data store (`notes.db` SQLite), the `uploads/` store, and the flows for `/notes`, `/upload`, `/files/<name>`; mark the Internet→app trust

boundary with a dashed line. *Deliverable:* DFD image embedded in your copy of the template.

(Task 1)



Task 2 — STRIDE the elements (30 min) · *Goal:* enumerate threats per element. *Steps:* for each element fill the S/T/R/I/D/E grid. Ground it in real code: `/notes` accepts a client-supplied `owner` with no auth (Spoofing); `/upload` saves raw `f.filename` — arbitrary-file-write (Tampering) — and echoes the resolved save path back in its response (Information disclosure); `/files/<name>` reads it back but is comparatively defended (see Task 5); no logging anywhere (Repudiation). *Deliverable:* completed STRIDE table.

(Task 2)

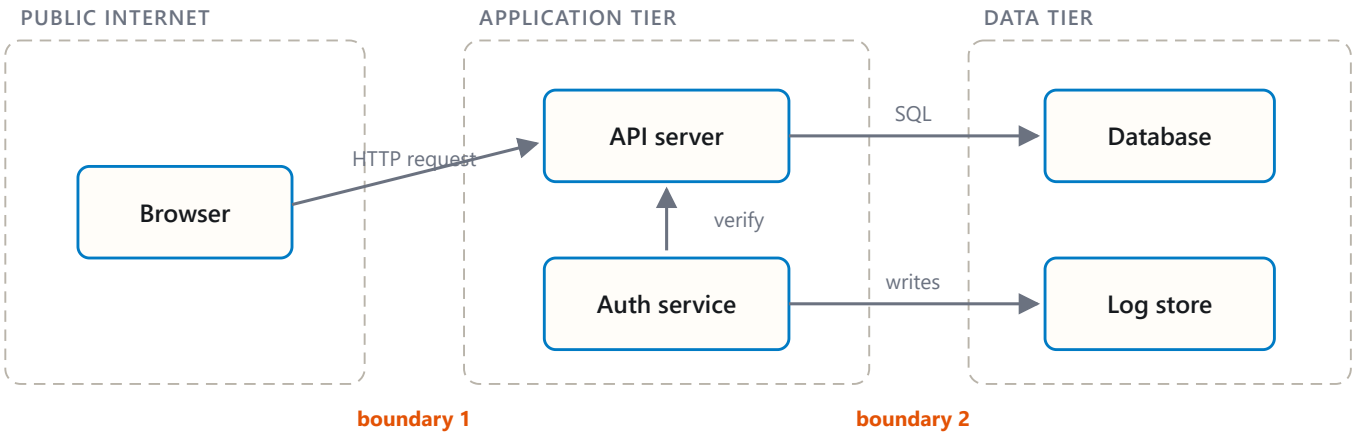
Element	S — Spoofing	T — Tampering	R — Repudiation	I — Info Disclosure	D — DoS	E — Elevation
`/notes`	**Yes** — client can choose `owner` with no authentication	Possible — attacker can submit arbitrary note content	**Yes** — no logging/audit trail	Possible — notes may be readable without proper access control	Possible — many requests could consume resources	Possible if missing authorization allows access as another user
`/upload`	Low	**Yes** — raw `f.filename` is used when saving files	**Yes** — uploads are not logged	**Yes** — response reveals the resolved save path	Possible — very large/many uploads may consume disk	Possible if arbitrary file write reaches sensitive locations
`/files/<name>`	Low	Low / comparatively	**Yes** — file access is not	Possible — files can be returned	Possible — repeated	Low / comparatively defended

Element	S — Spoofing	T — Tampering	R — Repudiation	I — Info Disclosure	D — DoS	E — Elevation
		defended	logged	to the requester	file requests consume resources	
Application overall	No authentication means identity cannot be trusted	Input/file handling can modify stored data	**Yes** — no logging anywhere	Errors/paths/data may expose internal information	No obvious rate limiting	Missing authentication/authorization may allow unauthorized actions

Task 3 — Elevation of Privilege game (20 min) · *Goal:* find threats you missed. *Steps:* play the EoP deck against your DFD; each card you can tie to a real element/flow scores a point; record every valid threat. No printer or scissors? Draw from the digital deck below instead — same 78 cards, same rule. *Deliverable:* list of carded threats + score.

```
eop-deck
```

Task 3b — Systems-level pass (25 min) 🔭 · *Goal:* find what the per-element grid cannot see. Tasks 2 and 3 enumerate threats **one element at a time**, and that is exactly where threat models are known to stop short — students taught STRIDE alone reliably identify component threats and *discount system-level ones* (Joshi et al., ASEE 2024). So do a second pass over the **whole** diagram:



A threat applies where data CROSSES a boundary — not inside a box.

- **Trust boundaries end-to-end.** Follow one request from the client to `notes.db` and back. List every boundary it crosses. Which crossing has no check on it?
- **Assume one element is fully owned.** Pick the Flask process, then the `uploads/` store. For each: what does the attacker now *reach* — not what is it, but where does it get them?
- **Chain two "low" findings.** Find two threats you or the EoP deck rated minor that combine into something you would not accept. Write the chain as `A → B → consequence`.
- **One-line system claim.** Finish: "Even if every element-level mitigation in Task 8 is implemented, this system still fails if ___."

Use the simulation below before you start — toggle a component to attacker-controlled and watch what it reaches:

```
trust-boundary
```

Deliverable: the boundary list, two owned-element reachability notes, one written chain, and the system claim.

(Task 3b)

1. Trust boundaries end-to-end

For a /notes request:

Client → Flask app → notes.db → Flask app → Client

Crossing Trust boundary Check? Client → Flask Public Internet → Application tier ❌ No authentication check Flask → notes.db Application tier → Data tier ⚠️ SQL is parameterized, but there is no authorization/ownership check notes.db → Flask Data tier → Application tier No user-level access-control check Flask → Client Application tier → Public Internet Response is returned directly

Main unchecked crossing: Public client → Flask. The client can supply owner itself, and the application trusts it without verifying identity.

2. Assume one element is fully owned

Flask process owned: The attacker can now reach notes.db, the uploads/ directory, application routes, request/response data, and anything else available with the Flask process's filesystem permissions.

uploads/ store owned: The attacker can control files later exposed through /files/, replace or poison uploaded content, and potentially consume storage, affecting users who retrieve those files.

3. Chain two findings

Client-controlled owner → no logging → forged notes cannot be reliably attributed

Consequence: An attacker can impersonate another owner and the system has little evidence showing who actually created the note.

Another strong chain from the upload path is:

Save raw filename → attacker-controlled file placement → overwrite/modify server-accessible data

4. One-line system claim

Even if every element-level mitigation in Task 8 is implemented, this system still fails if requests can cross the public-to-application trust boundary without reliable authentication and authorization.

Task 4 — Abuse cases & attacker personas (20 min) · *Goal:* think like specific adversaries. *Steps:* define 2 personas (e.g. a curious logged-in user; an anonymous internet attacker) and write 2 abuse cases each against the sample app, tied to DFD elements. *Deliverable:* 4 abuse cases.

(Task 4)

Abuse Cases & Attacker Personas

Persona 1 — Curious Logged-in User

A normal user who tries to access or modify data beyond what they should.

> 1.Spoof another note owner The user sends a POST request to /notes and changes the owner field to another person's name.

DFD element: Client → Flask /notes

> 2.Access another user's notes The user requests notes without proper authorization checks and may see data belonging to other users.

DFD element: Flask → notes.db

Persona 2 — Anonymous Internet Attacker

An external attacker with no legitimate account who sends malicious requests directly to the application.

> 3.Upload a malicious or misleading file The attacker sends a crafted filename to /upload because the application saves the raw f.filename.

DFD element: Client → Flask /upload → uploads/

> 4.Cause storage exhaustion The attacker repeatedly uploads large or numerous files until the server runs out of storage, causing denial of service. DFD element: /upload → uploads/ directory

Task 5 — Path-traversal deep-dive (25 min) · *Goal:* analyze the riskiest flow. *Steps:* trace `/upload` → `/files/<name>`; explain how `../` in a filename escapes `uploads/`; sketch the secure design (`secure_filename`, store outside web root, allow-list extensions). *Deliverable:* the data flow + secure-design note.

```
(Task 5)
"Path-Traversal Deep-Dive"
Data Flow

Client → /upload → Flask app → uploads/ directory → /files/<name> → Flask app → Client

The /upload endpoint uses the filename provided by the client. If an attacker uploads a file named
../test.txt, the path becomes:

>uploads/../test.txt

This resolves outside the uploads/ directory, so the attacker may write files to other locations that
the Flask process can access. The /files/<name> route is safer if it uses send_from_directory(), but
that does not prevent the unsafe file write during upload.

Secure-design note
A safer upload design should:

- Run the filename through secure_filename().
- Store uploads in a dedicated directory outside the web root.
- Allow only approved extensions such as .txt, .png, or .pdf.
- Generate a server-side filename instead of trusting the client's filename.
- Reject oversized files and unexpected file types.

```python
from werkzeug.utils import secure_filename
import uuid

ALLOWED = {"txt", "png", "pdf"}

name = secure_filename(f.filename)

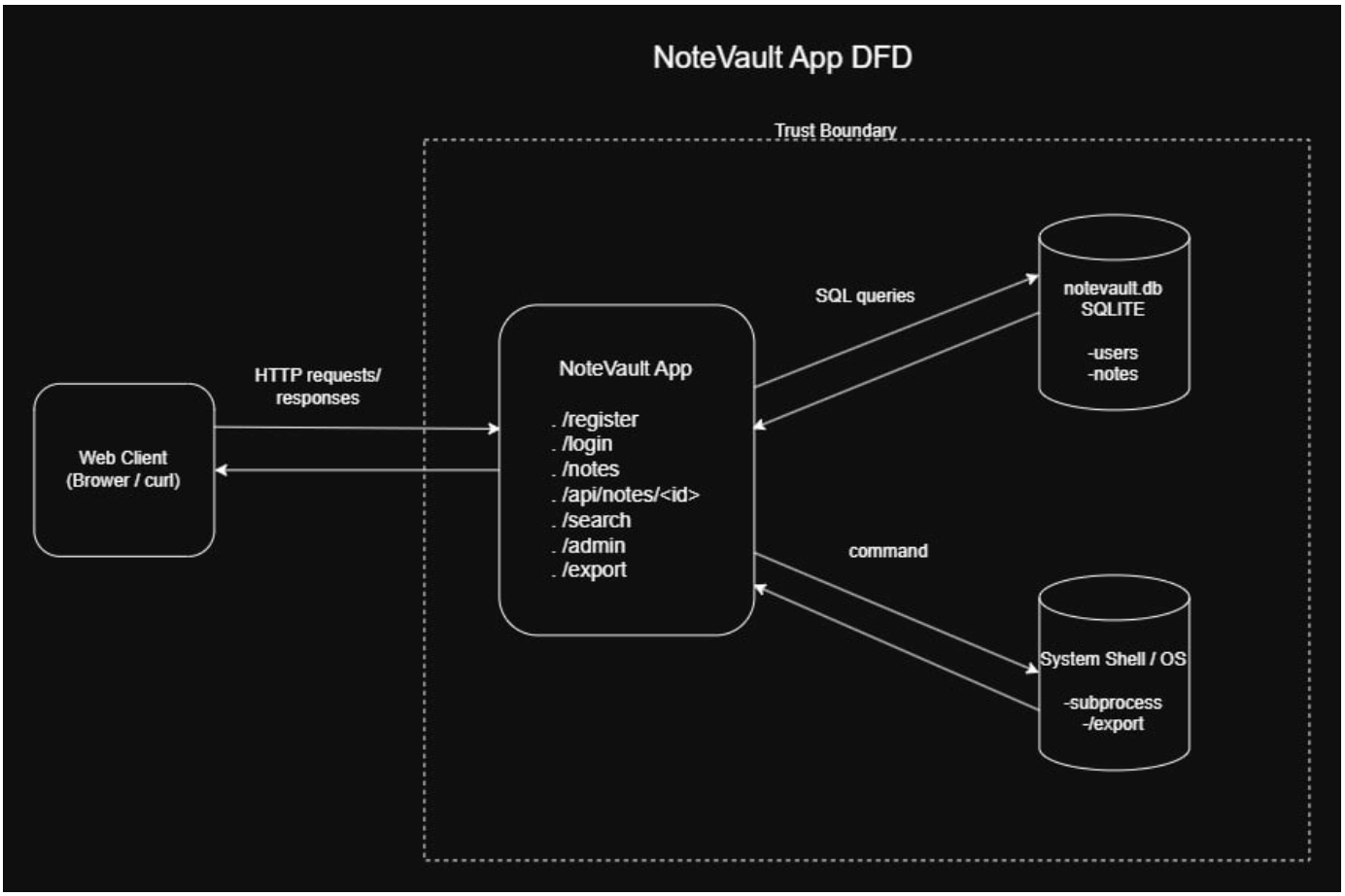
if "." not in name or name.rsplit(".", 1)[1].lower() not in ALLOWED:
    return "Invalid file type", 400

safe_name = f"{uuid.uuid4()}_{name}"
f.save(os.path.join(UPLOAD_DIR, safe_name))
```

Secure-design summary: Never use a client-supplied filename directly as a filesystem path; sanitize it, restrict file types, and give uploaded files server-controlled names.

Task 6 — Threat-model the project target (30 min) · *Goal:* kick off your term project. *Steps:* stop the sample-app first (`docker compose down` — both apps bind host port 8080), then run **NoteVault** (`cd ../../project/starter-app && docker compose up`), draw a quick DFD, and list the top 3 STRIDE threats you'd investigate. *Deliverable:* NoteVault DFD + top-3 threats (reuse these in your project report — `project/REPORT-TEMPLATE.md` in the repo root).
```

(Task 6)



Top 3 STRIDE Threats

Priority	STRIDE	Threat	DFD Element / Flow	Reason
1	**T — Tampering**	SQL injection through user-controlled input	Web Client → Flask App → SQLite DB (/login, /search)	User-controlled input is directly inserted into SQL queries, which could allow an attacker to change the intended database query.
2	**E — Elevation of Privilege**	User-controlled role during registration	Web Client → Flask App → SQLite DB (/register)	The registration endpoint accepts a client-supplied role, which could allow a user to assign themselves a higher-privileged role.
3	**E — Elevation of Privilege / T — Tampering**	Command injection through the export feature	Web Client → Flask App → System Shell (/export)	The fmt parameter is inserted directly into a shell command executed with shell=True, allowing untrusted input to influence an operating-system command.

Task 7 — Security requirements (15 min) · *Goal:* turn threats into testable requirements. *Steps:* write 3 security requirements as acceptance criteria ("the system must … so that …"), each mapped to a threat from Task 2 or Task 6. *Deliverable:* 3 testable security requirements.

#	Security requirement	Mapped threat
1	**The system must authenticate users and determine the note owner from the authenticated session, not from the client-supplied `owner` field, so that users cannot create notes while pretending to be another user.**	**Spoofing — `/notes`**
2	**The system must sanitize uploaded filenames and store files only inside the approved `uploads` directory, so that an attacker cannot use path traversal or crafted filenames to overwrite arbitrary files.**	**Tampering / Elevation of Privilege — `/upload`**

#	Security requirement	Mapped threat
3	**The system must log security-relevant actions such as note creation, file uploads, and failed requests with a timestamp and user identity, so that suspicious actions can be traced and users cannot easily deny performing them.**	**Repudiation — no logging**

Task 8 — Defend / fix it: rank & mitigate (25 min) 🛡️ · *Goal:* turn threats into action you can prove. *Steps:* rank the top 5 threats by likelihood × impact; propose one concrete mitigation each (e.g., auth on `/notes`, `secure_filename()` + allowlist for `/upload`, request logging for Repudiation, size/rate limits for DoS). Then **pick one and actually implement it** in your fork.

(Task 8)

Top 5 Threats — Rank & Mitigate

Rank	Threat	STRIDE	Likelihood	Impact	Risk	Concrete mitigation
1	`/upload` uses the user-supplied filename directly, allowing path traversal / arbitrary file write	Tampering / Elevation of Privilege	5	5	**25**	Use `secure_filename()`, allow only approved file types, and ensure the final path remains inside `UPLOAD_DIR`.
2	`/notes` trusts the client-supplied `owner`, so a user can pretend to be another user	Spoofing	5	4	**20**	Add authentication and obtain the owner identity from the authenticated session instead of request JSON.
3	`/notes` GET returns all stored notes without authentication or authorization	Information Disclosure	5	4	**20**	Require authentication and return only notes belonging to the authenticated user.
4	Uploads and requests have no size/rate limits, allowing resource exhaustion	Denial of Service	4	4	**16**	Add maximum upload/request sizes and rate limiting.
5	Important actions are not logged, so malicious users can deny what they did	Repudiation	4	3	**12**	Log security-relevant actions such as uploads, note creation, failures, timestamp, and authenticated user.

Selected Fix — Unsafe File Upload / Path Traversal

I selected the unsafe `/upload` endpoint because it used a client-controlled filename directly as part of a filesystem path. A filename containing `../` could escape the intended `uploads` directory and write a file elsewhere.

Before

```
@app.route("/upload", methods=["POST"])
def upload():
    f = request.files["file"]
    f.save(os.path.join(UPLOAD_DIR, f.filename))
    return {"saved": f.filename}
```

The original code joined `UPLOAD_DIR` directly with `f.filename`, which was supplied by the client.

After

```
@app.route("/upload", methods=["POST"])
def upload():
    uploaded_file = request.files.get("file")
```

```
    if uploaded_file is None or not uploaded_file.filename:
        return {"error": "File is required"}, 400

    original_name = uploaded_file.filename
    sanitized_name = secure_filename(original_name)

    if sanitized_name != original_name or not allowed_file(sanitized_name):
        return {"error": "Invalid filename or file type"}, 400

    stored_name = uuid.uuid4().hex
    uploaded_file.save(os.path.join(UPLOAD_DIR, stored_name))

    return {"saved": stored_name}, 201
```

The fixed endpoint rejects unsafe filenames and unsupported extensions. Accepted files are stored using a UUID generated by the server rather than the filename supplied by the client.

Before-fix evidence

Request:

```
curl.exe -s -X POST http://localhost:8080/upload -F "file=@$env:TEMP\week01-
proof.txt;filename=../escaped.txt"
```

Output:

```
{"saved":"../escaped.txt"}
```

The vulnerable endpoint accepted the traversal filename.

After-fix evidence

Same request:

```
curl.exe -i -X POST http://localhost:8080/upload -F "file=@$env:TEMP\week01-
proof.txt;filename=../escaped.txt"
```

Output:

```
HTTP/1.1 400 BAD REQUEST
{"error":"Invalid filename or file type"}
```

The fixed endpoint refused the same malicious request.

Valid-upload regression test

```
curl.exe -i -X POST http://localhost:8080/upload -F "file=@$env:TEMP\week01-
proof.txt;filename=proof.txt"
```

```
HTTP/1.1 201 CREATED
{"saved":"3c10336af0dd41d0b3257cd2f919c9f1"}
```

This confirms that approved uploads still work and receive a server-generated filename.

Git diff

- **Branch:** `week-01`
- **Commit:** `bcc28fe`
- **Commit message:** `Fix unsafe file upload path handling`

Why this closes the vulnerability class

This is a class fix for upload-path construction because no client-supplied filename becomes part of the stored filesystem path; every accepted file receives a server-generated UUID. It prevents all filename-based traversal patterns instead of blocking only the tested `../escaped.txt` value. An application-wide class fix would enforce the same rule through one shared storage function used by every endpoint that writes files.

Deliverable — the top-5 table, plus for the one you implemented:

1. the **diff** (commit hash on your `wk01` branch),
2. **evidence it works**: the request that succeeded before your change and is refused after — both outputs,
3. **why it closes the class, not the instance** (2–3 sentences). `secure_filename()` on one endpoint is an instance fix; "*no user-supplied string ever becomes a path component*" is a class fix. Say which yours is, and if it's an instance fix, say what the class fix would be.

> **Why this is weighted.** Fewer than half of working developers can spot a security hole in code, and being shown vulnerabilities does not by itself teach you to find or close them. Exploiting is the half that feels like progress; defending is the half that transfers to your job.

Part 4 — Reflection

1. Map your top finding to a CWE and to OWASP A06 (Insecure Design); explain the mapping in one sentence.

> 1.The unsafe /upload path is CWE-22: Path Traversal because external input was used to construct a path that could escape the intended directory; it maps to OWASP 2025 A06: Insecure Design because the upload feature was designed without a rule preventing client-controlled filenames from becoming storage paths. (

2. Name one real-world breach caused by a design flaw (not a missing patch) and what design control would have prevented it.

> 2.In the 2013 Target breach, attackers initially used an HVAC vendor’s credentials and later reached the point-of-sale network, affecting millions of customers. Strong network segmentation, least-privilege vendor access, and multi-factor authentication would have prevented or greatly limited this movement.

3. Of your five mitigations, which gives the most risk reduction per unit of effort, and why?

> 3.The upload-path mitigation gives the greatest risk reduction per unit of effort because a small code change blocks the highest-ranked threat: path traversal and arbitrary file writing. Rejecting unsafe filenames and using server-generated UUIDs removes client control over storage paths while allowing valid uploads to continue working.

Grading rubric (100)

Criterion	Points
Lecture questions (Part 2)	20
Exploitation + evidence (DFD + STRIDE table + EoP findings + screenshots)	40
Defense (top-5 ranking + mitigations)	25
Reflection (CWE/OWASP mapping + breach + best mitigation)	15

Assessed within the rows above (they are not extra points — they are what those points are for):

- **Systems-level reasoning** (inside *Exploitation + evidence*, Task 3b): does the model reach past single elements to boundaries, reachability and chains? Scored with the STRIDE + systems-thinking rubrics of Joshi et al. 2024.
- **Defensive proof** (inside *Defense*, Task 8): a claimed mitigation with no before/after evidence scores at most half. A mitigation you can show closing a *class* scores full.
- **Adversarial thinking** (across the whole sheet): do the abuse cases, personas and chains show you reasoning as an attacker with goals and constraints — or just listing categories? This is the course's central disposition and it is assessed, not assumed.

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
  1. What did you do, and **why did the vulnerability work**?
  2. **Why does your fix actually stop it** — and what could still break it?

Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.

AI assistant's full answer

> To prevent path traversal, sanitize the uploaded filename with Werkzeug's secure_filename() before saving it:

```
from werkzeug.utils import secure_filename

@app.route("/upload", methods=["POST"])
def upload():
    uploaded_file = request.files["file"]
    filename = secure_filename(uploaded_file.filename)
    uploaded_file.save(os.path.join(UPLOAD_DIR, filename))
    return {"saved": filename}
```

> secure_filename() removes dangerous path characters, so this completely secures the upload endpoint.

2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

What is wrong or risky

```
filename = secure_filename(uploaded_file.filename)
uploaded_file.save(os.path.join(UPLOAD_DIR, filename))
```

> The answer still uses a client-selected filename as the stored path, so two files with the same name could overwrite each other. It also does not reject an empty filename, restrict dangerous file types, or generate a unique server-controlled filename; therefore, the claim that it "completely secures" the endpoint is incorrect.

3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

Correct, verified version

```
import uuid

from werkzeug.utils import secure_filename

ALLOWED_EXTENSIONS = {"txt", "png", "pdf"}


def allowed_file(filename):
    return (
        "." in filename
        and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS
    )


@app.route("/upload", methods=["POST"])
def upload():
    uploaded_file = request.files.get("file")

    if uploaded_file is None or not uploaded_file.filename:
        return {"error": "File is required"}, 400

    original_name = uploaded_file.filename
    sanitized_name = secure_filename(original_name)

    if sanitized_name != original_name or not allowed_file(sanitized_name):
        return {"error": "Invalid filename or file type"}, 400

    stored_name = uuid.uuid4().hex
    uploaded_file.save(os.path.join(UPLOAD_DIR, stored_name))

    return {"saved": stored_name}, 201
```

> The AI's answer was insufficient because sanitization alone did not ensure that client-controlled filenames were kept out of storage paths. The corrected version rejects unsafe names and extensions and stores valid files under server-generated UUIDs. Verification showed that ../escaped.txt received 400 BAD REQUEST, while the valid proof.txt upload received 201 CREATED.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

> The /upload endpoint receives a file and originally saved it using the filename supplied by the user. An attacker could include ../ in that filename, causing the final path to leave the intended uploads folder and write a file somewhere else that the application can access.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Final prompt given to the AI:

> Fix only the path-traversal vulnerability in the Flask /upload endpoint. Read the file with request.files.get("file"); reject missing or empty filenames with HTTP 400; use secure_filename() and reject the request if sanitization changes the original filename; allow only txt, png, and pdf; and store accepted files using uuid.uuid4().hex so no user-supplied filename becomes part of the filesystem path. Preserve the route, save inside UPLOAD_DIR, return HTTP 201 for a valid upload, and provide complete Python code including required imports and helper functions.

Verified result:

> I ran the original attack again using the filename ../escaped.txt. The fixed endpoint returned 400 BAD REQUEST with {"error":"Invalid filename or file type"}, so the path-traversal exploit failed. A normal upload named proof.txt still returned 201 CREATED and was stored under a server-generated UUID.